



MFE113TC

Programming Foundation for Intelligent Manufacturing Engineering

Lecture 07 Data Structure in C/C++

Dr. Jiachen Liu
jiachen.liu@xjtlu.edu.cn

Fall, 2026



Outline

- Multi-dimensional Arrays / Pointers
- Dynamic Memory Allocation in C and C++

Multi-Dimensional Arrays and Pointers

Pointer to an array

- So we can use pointers to deal with 1D arrays
- What will happen when we have a 2D array?

```
int table[4][3];    /* declares a 2D array */
```

-> **the name of the array is a constant pointer**, where the address of the pointer cannot be changed.

- We can think of this as a **4 element array, where each element is another 3 element array**;
- How would we declare a pointer to this table?
 - `int (*ptable)[3];`
 - The pointer points **to an array** rather than an **int**.
 - Be noted that **()** is used, since **[]** is with **higher precedence than ***.

Multi-Dimensional Arrays and Pointers

2D Arrays and Pointers

E.g., `int table[4][3];` */* an array with 4 elements */*

/ each element is an array with 3 elements */*

- `table[0]` is the address `&table[0][0]`;
- Then `(table[0]+2)` is the address `&table[0][2]`;
- And `(table[2]+1)` is the address `&table[2][1]`;
- Using the symbol ***** to get access to the value of these elements;

<code>table[0]</code>		<code>table[0][0]</code>	<code>table[0][1]</code>	<code>table[0][2]</code>
<code>table[1]</code>		<code>table[1][0]</code>	<code>table[1][1]</code>	<code>table[1][2]</code>
<code>table[2]</code>		<code>table[2][0]</code>	<code>table[2][1]</code>	<code>table[2][2]</code>
<code>table[3]</code>		<code>table[3][0]</code>	<code>table[3][1]</code>	<code>table[3][2]</code>

Multi-Dimensional Arrays and Pointers

2D Arrays and Pointers

E.g., `int table[4][3];` */* an array with 4 elements */*

/ each element is an array with 3 elements */*

- `table[2]` is the address `&table[2][0]` —> **pointing to the whole 3rd row**;
 - a pointer of type `int*`;
- `table+2` is now the address `&table[2]`;
- `table[2] == *(table+2)`; `*(table+2)` takes the value in `table[2]`, which is `&table[2][0]`
 - **2-d array is actually a pointer of pointers, saving the addresses**;
 - **`*(table+2)` accesses the `table[2]`, which further decays to `&table[2][0]`**

<code>table[0]</code>		<code>table[0][0]</code>	<code>table[0][1]</code>	<code>table[0][2]</code>
<code>table[1]</code>		<code>table[1][0]</code>	<code>table[1][1]</code>	<code>table[1][2]</code>
<code>table[2]</code>		<code>table[2][0]</code>	<code>table[2][1]</code>	<code>table[2][2]</code>
<code>table[3]</code>		<code>table[3][0]</code>	<code>table[3][1]</code>	<code>table[3][2]</code>

Multi-Dimensional Arrays and Pointers

2D Arrays and Pointers

E.g., `int table[4][3];` */* an array with 4 elements */*

/ each element is a an array with 3 elements */*

- How would you address `&table[2][1]`?
 - `*(table+2)+1`
- How would the element `table[2][1]` be accessed?
 - `*(*(table+2)+1)`

<code>table[0]</code>		<code>table[0][0]</code>	<code>table[0][1]</code>	<code>table[0][2]</code>
<code>table[1]</code>		<code>table[1][0]</code>	<code>table[1][1]</code>	<code>table[1][2]</code>
<code>table[2]</code>		<code>table[2][0]</code>	<code>table[2][1]</code>	<code>table[2][2]</code>
<code>table[3]</code>		<code>table[3][0]</code>	<code>table[3][1]</code>	<code>table[3][2]</code>

Multi-Dimensional Arrays and Pointers

2D Arrays and Pointers

- Addressing / Accessing Summaries: supposing $x[a][b]$

Exercise

Locations to be addressed	Addressing Get the Addresss	Accessing Get the Value
$x[0][0]$	?	?
$x[0][1]$	?	?
$x[1][0]$	?	?
$x[1][2]$	?	?

Multi-Dimensional Arrays and Pointers

2D Arrays and Pointers

- Addressing / Accessing Summaries: supposing $x[a][b]$

Locations to be addressed	Addressing Get the Addresss	Accessing Get the Value
$x[0][0]$	$*(x)+0$	$*((*x)+0) / **x$
$x[0][1]$	$*(x+0)+1$	$*((*x+0)+1) / *((*x)+1)$
$x[1][0]$	$*(x+1)+0$	$*(*(x+1)+0) / ** (x+1)$
$x[1][2]$	$*(x+1)+2$	$*(*(x+1)+2)$



Multi-Dimensional Arrays and Pointers

2D Arrays and Pointers

- Considering the following:

float table[M][N];

- In general, for the n-th element of the m-th array:

$*(*(table+m)+n) == table[m][n];$

$((*(table+m))[n] == table[m][n])$

Multi-Dimensional Arrays and Pointers

2D Arrays and Pointers

- Considering the following:

```
int table[4][3];
```

```
int *ptr;
```

What are the following commands referring to?

- a). `ptr = table[1];` `/* &table[1][0] */`
- b). `ptr = table[2][1];` `/* incorrect */`
- c). `ptr = table[2];` `/* &table[2][0] */`
- d). `ptr = table+2;` `/* &table[2], incorrect */`
- e). `ptr = *table+2;` `/* &table[0][2] */`

In d)., Solely using `table+2` is ok! It is `&table[2]`, however:

- `ptr` has type `int *`, while `table` decays to a pointer to an array of 3 integers, i.e. `int (*)[3]`. Therefore, `ptr = table + 2;` involves incompatible pointer types.**

Multi-Dimensional Arrays and Pointers

Shallow Copy & Deep Copy

- **Shallow copy:** only copy the address, E.g.:

```
int a = 0;
```

```
int *p1 = &a, *p2;
```

```
p2 = p1          /* only copy the address of p1 to p2 */
```

- **Deep copy:** copy each individual element of the object;

```
int a[5] = {1,2,3,4,5}, b={-1,-1,-1,-1,-1};
```

```
b[0]=a[0], b[1]=a[1], b[2]=a[2], b[3]=a[3], b[4]=a[4];
```

```
/* copy every individual element */
```

Multi-Dimensional Arrays and Pointers

Array of pointers

- So we have looked at arrays and how they relate to pointers. But now,

how about an array of pointers?

```
int* table[3];
```

- This is an array with 3 elements and **each element is an int pointer**;

```
int (*table)[3];
```

- This is a **single pointer** that points to an **int** array with 3 elements.

Multi-Dimensional Arrays and Pointers

Quiz

- Which of the following commands declare and initialise an array containing the days Saturday and Sunday?
 - a). `char*` days = {"Saturday", "Sunday"};
 - b). `char*` days[2] = {"Saturday", "Sunday"};
 - c). `char` days[2] = {"Saturday", "Sunday"};

Multi-Dimensional Arrays and Pointers

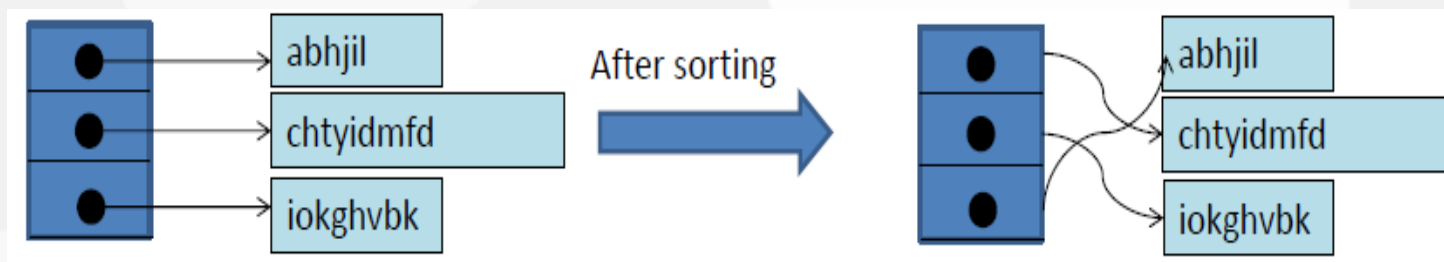
Quiz

- Which of the following commands declare and initialise an array containing the days Saturday and Sunday?
 - a). `char* days = {"Saturday", "Sunday"};`
 - b). `char* days[2] = {"Saturday", "Sunday"};`
 - c). `char days[2] = {"Saturday", "Sunday"};`

Multi-Dimensional Arrays and Pointers

Array of pointers

- Why do we want an array of pointers?
 - Better use of memory space;
 - **Processing efficiency.**



- During ordering, in the end we **don't need reposition the values** in the RAM;
- What we only need to do is to **update the address saved in the pointer.**

Multi-Dimensional Arrays and Pointers

Pointer to Pointer

- How do we declare a pointer to another pointer?

```
int *p1;    /*a pointer to an int named p1*/  
int **p2;   /*a pointer to an int pointer named p2*/  
p2=&p1;     /*so p2 could hold the address of p1*/
```

- For sending and returning pointers to functions
- Remember: The pointer and array should be **matched** at the same level:

2D Array $\longleftrightarrow$ pointer to pointer

1D Array $\longleftrightarrow$ pointer

Multi-Dimensional Arrays and Pointers

Pointer to Pointer

```
int table[4][3], myarray[4][3];  
int *table1[3];  
int **pTable;  
int *Ptr1;  
int (*pPtr)[3];
```

int * = int *

int * != int (*)[3]

int ** = int **

int * = int *

int * = int *

int ** != int *

int (*)[3] = int (*)[3]

*pTable=table[1]; **/*Valid, table[1] indicates the array of 2nd row*/**

Ptr1=table+1; **/*Invalid, the level of pointer and array should match*/**

pTable=table1; **/*Valid*/** &table[1][0]

Ptr1=table[1]; **/*Valid*/**

Ptr1 =*(table+1); **/*Same Valid as above*/**

pTable=table[1]; **/*Invalid*/**

table= myarray; **/*Invalid*/**

pPtr =table; **/*Valid*/**

Array names are not modifiable
lvalues; arrays cannot be assigned
using =; array itself is not assignable
pPtr = &table[0];



Multi-Dimensional Arrays and Pointers

2D Array Function Prototype

- Functions that process *multi-dimensional* arrays needs to be correctly declared with the function arguments.
- But how to write a function prototype when we pass it a *multi-dimensional* array as an argument?

Multi-Dimensional Arrays and Pointers

2D Array Function Prototype

When passing a 2D array to a function, the column size must usually be specified.

`int ar[4][3];`  `void sum(int ar[][3], int rows);` **OR** `void sum(int (*ar)[3], int rows);`

void sum(**int** ar[][3], **int** rows);



void sum(**int** (*ar)[3], **int** rows);



These two prototypes are equivalent for a 2D array with 3 columns.

void sum(**int** **ar, **int** rows, **int** cols);



BUT, `int **` is a different representation and is not directly compatible with `int table[4][3]`. In some cases, it cannot be used interchangeably.

Multi-Dimensional Arrays and Pointers

2D Array Function Prototype

- Example 1: with the main function (only for demonstration; code is incomplete)

```
#include <stdio.h>

#define COLS 2
#define ROWS 3

double sum2d(double (*ar)[COLS], int rows); /*prototype*/

main () {
    double ar[ROWS][COLS] = { 1.2, 3.2, 4.9, 3.0, 23.9, 18.7 };
    double total;
    total = sum2d(ar,ROWS);          /*call function*/
    printf("Sum of all elements is %lf",total);
}
```

Multi-Dimensional Arrays and Pointers

2D Array Function Prototype

- Example 1: function sum2d

```
double sum2d(double (*ar)[COLS], int rows) {  
    int i, j;  
    double tot = 0;  
    for(i=0; i<rows; i++)          /*nested loops for 2d array*/  
        for(j=0; j<COLS; j++)  
            tot += ar[i][j]; /*sum elements one by one*/  
    return tot;  
}
```

Vectors in C++

Vector Fundamentals

- Recall the Linked List

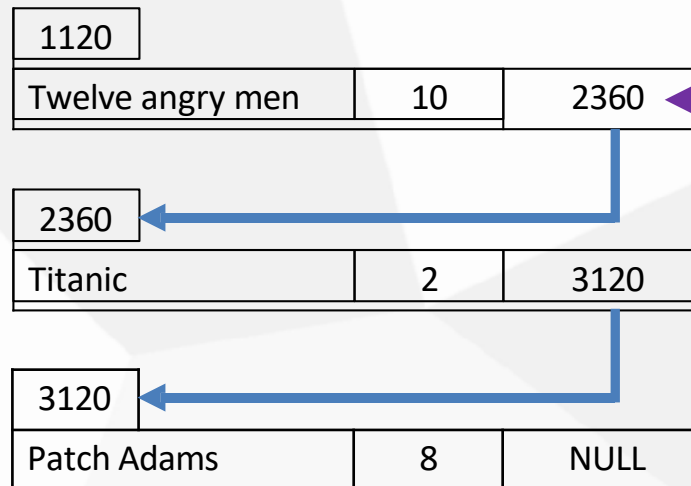
```
struct movie{  
    char * title;  
    int rating;  
    struct movie *next;  
};
```

Pointer to first
structure

1120



Lamb Shashlik



Pointer to next
structure

Feature	C Array	C++ vector
Size	Fixed after declaration	Can grow or shrink dynamically
Example	<code>int a[5];</code>	<code>vector<int> v;</code>
Element access	<code>a[i]</code>	<code>v[i]</code> or <code>v.at(i)</code>
Get number of elements	No built-in <code>.size()</code>	<code>v.size()</code>
Add elements	Not directly supported	<code>v.push_back(value)</code>
Remove elements	Not directly supported	<code>v.pop_back()</code>
First element	<code>a[0]</code>	<code>v.front()</code>
Last element	<code>a[n-1]</code>	<code>v.back()</code>
Memory management	More manual	Automatically managed
Bounds checking	No	<code>at()</code> provides bounds checking
Resizing	Not supported directly	Supported
Language	C and C++	C++ only

Vectors in C++

Vector Declaration

- Another way: using the **<vector>**:
 - To use the vector, at the beginning, **#include <vector>**
 - Declaration:
 - Syntax: `vector<dataType> name(size);`
 - E.g.:

```
vector<int> iArr(10);           // a vector with 10 elements

int v_size = 5;
vector<Complex> cNum(v_size);  // create the vector
                                // with the variable
                                // is also valid
                                User-defined

vector<int> empty;             // valid as an empty vector
```

But you can change it later!

Vectors in C++

Vector Initialization

- Another way: using the **<vector>**:
 - To use the vector, at the beginning, **#include <vector>**
 - Initialization:
 - Can be initialised as normal array

```
vector<int> initial(3) = {1, 2, 3}; // Invalid
vector<int> iVec(5, 1);             // Valid, 5 elements are all 1
int arr[] = {1, 2, 3};
vector<int> iTial(arr, arr+3);      // Valid, initialize based on arr,
                                   // from element 0 to element 2
```

Vectors in C++

Vector **Accessing**

- Accessing the vector
 - The same as normal arrays, using **[] and indices**;
E.g.: `iVec[0] = 10;` // the first element is assigned to be 10
`iVec.at(0) = 10;` // also valid
 - `Vec_name.front()`: **returns the first element of the vector**;
 - `Vec_name.back()`: returns the **last** element of the vector;
E.g.: `cout << iVec.front();`
`iVec.front() = iVec.front() - iVec.back();`
 - **Get the size of the vector**:
 - Get the number of elements in the vector container (`vector::size_type`; `size_t`; unsigned int)
 - E.g., `cout << iVec.size();`
`for (unsigned int i=0; i<iVec.size();i++) {.....}`
`for (size_t i = 0; i < iVec.size(); i++) {.....}`

Vectors in C++

Vector Manipulation

- Manipulating the vector

- To remove an element in an ordered vector

- Move** all elements following the element to be removed down (to a lower index) by one slot;
- then **delete** the last element;
- Code: `pos = 2;`

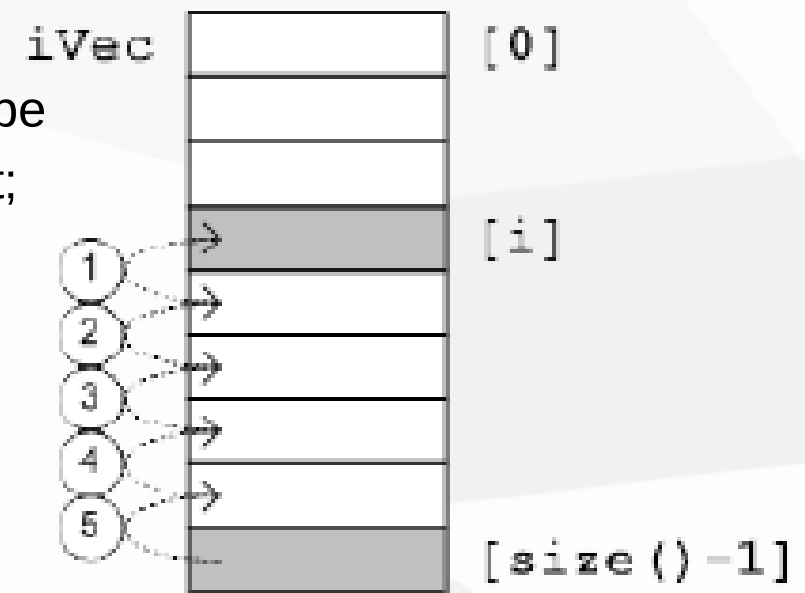
```
for (int i=pos; i<values.size()-1; i++)  
    values[i]=values[i+1];  
values.pop_back();
```

`values.pop_back()` means delete the last element

- Q: consider how to **insert** an element to a specific position without influencing the order of other elements.

`push_back()` means add one element to the end

[10][20][30][40][50] → [10][20][40][40][50]



Vectors in C++

Vector as Arguments

- Passing a vector into a function **by value**
 - The function makes a **local copy** of all the content of the vector;
 - Syntax: **return_type** function_name(**vector<type>** vet_name);
 - Example: a function computing the averaging of all elements

local copy

```
double average(vector<double> values)
{
    double sum = 0;
    for (unsigned int i=0; i<values.size(); i++)
        sum = sum + values[i];
    return sum/values.size();
}
```

**No affect
on original
vector**

```
int main()
{
    double test[5]={1.2,-2.4,3,4.02,0};
    vector<double> value(test, test+5);
    cout <<"The average is "<<average(value)<<endl;
    return 0;
}
```

Vectors in C++

Vector as Arguments

- Passing a vector into a function **by reference**
 - The function can **modify the content of the vector**;
 - Syntax: **return_type** function_name(**vector<type>&** vet_name);
 - Example: a function raising all elements by the given rate

**Directly
modify the
original
vector**

```
void raise(vector<double>& values, double rate)
{   for (unsigned int i=0; i<values.size(); i++)
    values[i]=values[i]*(1+rate/100);
    // rate is in percentage
}

int main()
{   vector<double> savings(5);
    for (unsigned int i=0; i<savings.size(); i++)
        savings[i]=i*200;    // initialise
    raise(savings,5.2);      // call the function
    return 0;
}
```

{0, 210.4,
420.8, 631.2,
841.6}

{0, 200, 400, 600, 800}

Vectors in C++

Vector as Return Value

- Using the vector as the function **return value**:
 - The function can return a vector;
 - Syntax: **return_type** function_name(param list);
 - Example: a function that collects all values that will fall within a given range

push_back():
add new
element to the
vector end

```
vector<double> between(vector<double> values, double low, double high)
{
    vector<double> result;
    for (unsigned int i=0; i<values.size(); i++)
        if (low<=values[i] && values[i]<=high)
            result.push_back(values[i]);
    return result;
}

int main()
{
    values = {0.5, 1.2, 2.5, 4.0, 3.0};
    ... // initialise a test vector
    vector<double> ranged = between(values, 1, 3);
    ... // output the values for check
    return 0;
}
```

Dynamic Memory Allocation in C

Why we need dynamically allocate memories?

- The compiler allocates enough memory for your whole program including variables, arrays, and functions;
- But **what happens if you need more memory space during program execution?**
 - C allows for **memory to be allocated at runtime** i.e. you can add memory space on demand (as long as it is available and possible ...)
- For simple programs, dynamic memory allocation may not be used very often. However, it becomes important when the amount of memory needed is only known at runtime. e.g., user input.

Dynamic Memory Allocation in C

The malloc() function

- The function malloc() requests a **block of memory** to be made available of **a specified size**;
- The return value of the function is **a pointer to the newly available memory**;
- The type of the pointer is **generic**:
 - It needs to be **typecast to the required type** (e.g. int, float, char, etc...);
- Be noted, the function malloc() is contained in the header file **stdlib.h**

Dynamic Memory Allocation in C

The malloc() function

- The function malloc() will **find a suitable block of free memory** and **returns the address of the first byte**;
- That address is **assigned to a pointer variable (ptr)**;
 - The pointer is used to **access the memory**
- Typecast is needed to make sure that the ptr type and malloc() return type are the same.

```
ptr = (double*) malloc(6*sizeof(double));
```

Note the typecast (double * ptr)

OR double *ptr = malloc(6 * sizeof(double));

Input the parameter:
size of the memory needed
6-double

$6 \times 8 = 48$ bytes

Dynamic Memory Allocation in C

The malloc() function

- If the function fails to obtain the memory, **NULL** ('\0') will be returned;
- Usually, the function malloc() is used with a **test for failure**:

```
if (ptr == NULL)
{
    printf ("Memory allocation Failed");
    return 1;
}
```

Dynamic Memory Allocation in C

The free() function

- To release memory allocated with the function malloc(), use:

```
free(ptr);
```

- It frees the block of memory starting at the address stored in ptr;
- It should **ONLY** be used where the memory block was allocated by the function malloc();
- You **CANNOT** use the function free() to free memory allocated by other means;
- Note: the function free() is also contained in the header **stdlib.h**

Dynamic Memory Allocation in C

The free() function

- **Always** use free() when you finished using the memory allocated with malloc();
 - —> They are occurring as **pairs**;
- **VERY IMPORTANT**: if you allocate memory **repeatedly** and **do not free any of them**, you will run **Out Of Memory (OOM)**;
 - E.g., in the for loop
 - This will cause the program to crash...
(where the pointer returned by malloc has been modified...)

```
(double*) ptr;  
for (ii=0;ii<5;ii++)  
    ptr = (double*)malloc(5*sizeof(double));
```

Dynamic Memory Allocation in C

Create dynamically sized array

- Two major ways to create an array:
 - Declare an array using **constant** expressions for the array dimension i.e. `int array[10]`.
 - Once the declared size of the corresponding array is fixed, it **cannot be resized at runtime**;
 - Declare a pointer then call `malloc()` with the number of elements you want to at the runtime.
 - The pointer can be used to access the elements of the array.
 - The length of the array is then flexible each time you run the program;
 - The array is now to **be dynamically sized**.

Dynamic Memory Allocation in C

Create dynamically sized array

```
#include <stdio.h>
#include<stdlib.h> /*malloc() and free()*/
int main(){
    double *ptd;
    int max, number, i=0;
    puts("How many doubles do you want to enter?");
    scanf("%d",&max);
    ptd = (double *)malloc(max * sizeof(double));
    if(ptd==NULL){
        puts("Memory allocation failed. Goodbye.");
        return 0;
    }
    /*ptd now points to an array of doubles of length max*/
```

Dynamic Memory Allocation in C

Summary

```
ptr = (double*) malloc(6*sizeof(double));
```



Note the typecast (double * ptr)



Input the parameter:
size of the memory needed
6-double

$6 \times 8 = 48$ bytes

```
free(ptr);
```

Dynamic Memory Allocation in C++

Operator **new**

- The operator **new** allocate memory for a particular type of data and return the memory address;
- Syntax: new dataType;
 - E.g.: **double*** pd1;
pd1 = **new double**;
*pd1 = 100.465;
delete pd1;
- At this point, a double variable has been **dynamically allocated in memory**, but the variable name is unknown, since pd1 is only a pointer.
- This variable can only be accessed through a pointer.

Dynamic Memory Allocation in C++

Dynamic Allocation of an Array

- Syntax: `double* pd2 = new double[n];`
 - Here, the integer `n` **specifies the length of the array that you want to dynamically allocate**. The value can be changed during the execution.
- Examples:

```
*pd2=1000.02;           // first element of array  
*(pd2+1)=2435.08;       // second element of array  
*(pd2+n-1)=43552.55;    // nth element
```

Dynamic Memory Allocation in C++

To verify successful Memory Space Allocation ?

- `pd2 = new double[n];`
 - depending on the returning value:
 - returns 0, meaning **no memory** space left;
 - returns **non-zero**, meaning the allocation is successful.

```
if(!(pd2 = new double[n]))  
{  
    cout<<"not enough memory space"<<endl;  
    exit(1);  
}
```



Dynamic Memory Allocation in C++

A Example

```
#include<iostream>
using namespace std;
int main(void)
{
    int num;
    cout << "Type in the number of real values you are going to input from the keyboard" << endl;
    cin >> num; // The number of values to be input

    float *pvalue = new float [num]; //Allocate float array
    for (int i=0; i<num; i++)
    {
        cout<<"Key in value"<<(i+1)<<" Please : " << endl ;
        cin>>*(pvalue+i); // Read in the ith value
    }

    .....
    delete [ ] pvalue; //De-allocate memory after use
}
```

Comparioson

C	C++
malloc()	new
malloc(n*sizeof(double))	new double[n]
free(ptr)	delete ptr / delete[] ptr



Dynamic Memory Allocation in C++

Summary

- In OOP(Object-Oriented Programming), use **new** to dynamically allocate memory during processing the program
- The **size of the dynamic memory allocation** can be a **variable** (with value)
- The memory allocated by **new** can only be used by the specified program, so it must be de-allocated by **delete** before the end of the program. Otherwise, it will cause memory leak
- For dynamic array, the format of delete must be **delete []**. Otherwise, it still causes memory leak.



Thanks